

BULK METER READING - MOBILE APPLICATION DEVELOPMENT

1. EXECUTIVE SUMMARY

This development progress report outlines the current implementation milestones of the **EDL Bulk Meter Reading Mobile Application**. Designed to transition billing operations from manual paper entry to offline-first synchronization, the application features local secure session authentication and local SQLite caching to perform seamlessly in remote areas. Development is currently focused on **Step 3: Download Pending Readings**.

Overall Project Completion

40%



Phase 1 (Setup) & Phase 2 (Authentication) are fully optimized. Phase 3 is underway.

Roadmap Status

Step 3 of 6

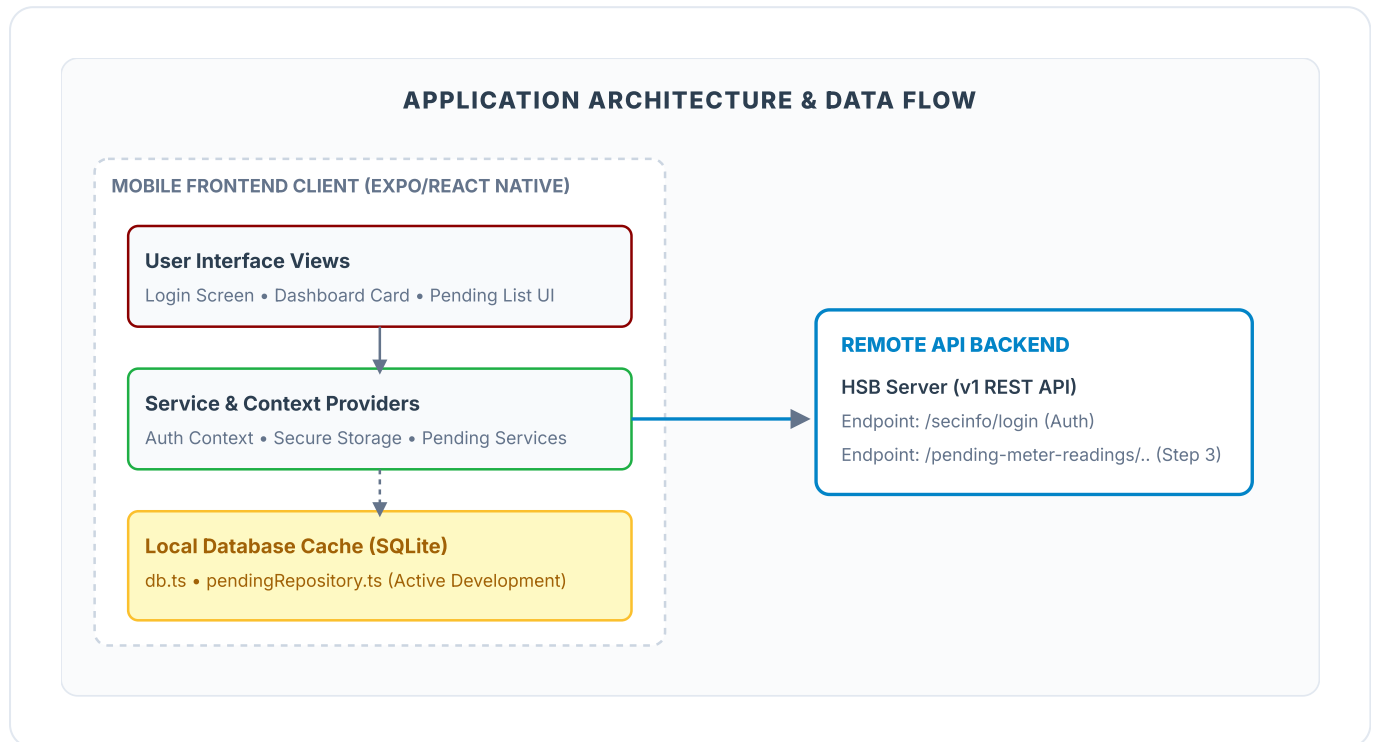
Current Focus: Syncing remote API endpoints with the local SQLite data warehouse to allow offline reader capabilities.

2. ROADMAP PROGRESSION

STEP	MILESTONE	STATUS	IMPLEMENTATION DETAILS
1	Project Setup	COMPLETED	Expo SDK 54 initialization, File-based routers, SQLite initialization structure.
2	Authentication	COMPLETED	API-based sign-in with Secure Store session mapping. Automatic login route guards.
3	Download Pending Readings	IN PROGRESS	API consumption is ready. Caching and mapping data into SQLite is under development.
4	Reading Capture	PENDING	UI layout for customer accounts, reading insertion form, and client-side validations.
5	Synchronization	PENDING	Uploading local records, conflict handling, and marking synchronization statuses.
6	Polish	PENDING	Network listeners, search queries, pull-to-refresh feeds, and UI enhancement.

3. SYSTEM ARCHITECTURE

The application employs a secure, modular model-view-service structure. Secure operations and initial layouts are governed by context hooks. Local operations read/write from a local cache database to maintain full usability during network drops.



4. TECHNICAL IMPLEMENTATION CHECKLIST

✓ Secure Storage Implementation

Integrates `expo-secure-store` to cache and read user credentials locally without plain text leaks.

✓ Authentication Redirection Layouts

Guards the application views. Logged out users are forced to root, while authenticated sessions land on the main dashboard.

✓ Axios API Client Layer

Centralized axios networking client utilizing interceptors and base configurations matching version 1 API expectations.

⌚ Offline Cache Infrastructure

Structuring repository tables mapping API responses directly to SQLite queries. Currently setting up database migrations.

○ Meter Reading Validation Rules

Form components with limits enforcing values ensuring current values does not fall lower than previous indices.

○ Auto Synchronization Scheduler

Service task capturing dirty rows inside local SQLite schema and uploading items to bulk servers when network is up.

5. NEXT IMMEDIATE OBJECTIVES

To successfully pass Step 3 and execute Step 4:

- Define the SQLite schema in `db.ts` to map the downloaded `PendingReading` properties.
- Write bulk-insert methods in `pendingRepository.ts` to persist network payloads immediately.
- Refactor the Pending Readings Screen to query SQLite first, and add a pull-to-refresh network sync option.
- Design Step 4's reading entry input modal with validation logical rules.